UNIT IV – C++ DEBUGGING SOLUTIONS

Inheritance & Pointers

Solutions to the five debugging problems in the uploaded Unit IV question bank. Compiler diagnostics may vary slightly by compiler.

Question 1: Diamond Problem and Virtual Inheritance

Bug and root cause

The ambiguous access is at ta.idNumber (the question labels this Line 33). Student and Employee each inherit Person non-virtually, so TeachingAssistant contains two separate Person subobjects and therefore two idNumber fields. A typical diagnostic is: “request for member ‘idNumber’ is ambiguous.”

Fix: inherit Person virtually in both intermediate classes. In a virtual-base diamond, the most-derived class TeachingAssistant is responsible for constructing the shared Person subobject; it must directly initialize Person.

Corrected code

```
#include <iostream>

class Person {
public:
    int idNumber;
    Person(int id = 0) : idNumber(id) {
        std::cout << "Person initialized: " << idNumber << std::endl;
    }
};

class Student : virtual public Person {
public:
    Student(int id) : Person(id) {}
};

class Employee : virtual public Person {
public:
    Employee(int id) : Person(id) {}
};

class TeachingAssistant : public Student, public Employee {
public:
    TeachingAssistant(int sId, int eId, int personId)
        : Person(personId), Student(sId), Employee(eId) {}
};

int main() {
    TeachingAssistant ta(101, 202, 303);
    std::cout << "TA ID: " << ta.idNumber << std::endl;
}
```

Output:

```
Person initialized: 303
TA ID: 303
```

The Person initializers inside Student and Employee are ignored when those classes are subobjects of TeachingAssistant; the most-derived initializer wins.

Question 2: Missing Base Constructor Call

Bug and root cause

Device has only Device(std::string), not a default constructor. Sensor's constructor omits Device(...) from its initializer list, so the compiler attempts Device() before entering the Sensor constructor body. That call is invalid. Assigning macAddress in the body is too late: the base subobject must already be constructed.

Construction order: base class Device first, then Sensor members, then the Sensor constructor body. Use Sensor(...): Device(mac), samplingRate(rate).

Corrected code

```
#include <iostream>
#include <string>

class Device {
protected:
    std::string macAddress;
public:
    Device(std::string mac) : macAddress(mac) {
        std::cout << "Device configured with MAC: "
                  << macAddress << std::endl;
    }
};

class Sensor : public Device {
private:
    int samplingRate;
public:
    Sensor(std::string mac, int rate)
        : Device(mac), samplingRate(rate) {}

    void display() const {
        std::cout << "MAC: " << macAddress
                  << ", Rate: " << samplingRate << " Hz" << std::endl;
    }
};

int main() {
    Sensor tempSensor("00:1A:2B:3C:4D:5E", 100);
    tempSensor.display();
}
```

Output:

```
Device configured with MAC: 00:1A:2B:3C:4D:5E
MAC: 00:1A:2B:3C:4D:5E, Rate: 100 Hz
```

Question 3: Base Pointer Access and Object Slicing

Bug and root cause

Vehicle* has static type Vehicle*, so member access is checked against Vehicle's declarations. Even if it points to a SportsCar, engageTurbo() is not declared in Vehicle and cannot be called directly. Object slicing happens when SportsCar is copied into a Vehicle value: only the Vehicle base portion is copied, and turboEnabled is discarded.

Use a checked downcast (dynamic_cast requires a polymorphic base; this question's Vehicle is not polymorphic) or an appropriate static_cast when the actual object type is known. Use references/pointers rather than copying derived objects into base values to avoid slicing. This corrected example adds a virtual destructor to make dynamic_cast valid.

Corrected code

```
#include <iostream>

class Vehicle {
public:
    int maxSpeed;
    Vehicle(int s) : maxSpeed(s) {}
    virtual ~Vehicle() = default;
    void honk() { std::cout << "General Horn" << std::endl; }
};

class SportsCar : public Vehicle {
public:
    bool turboEnabled;
    SportsCar(int s, bool turbo)
        : Vehicle(s), turboEnabled(turbo) {}
    void engageTurbo() {
        std::cout << "Turbo Engaged!" << std::endl;
    }
};

int main() {
    Vehicle* vPtr = new SportsCar(300, true);
    vPtr->honk();

    if (SportsCar* sportsPtr = dynamic_cast<SportsCar*>(vPtr))
        sportsPtr->engageTurbo();

    SportsCar car(250, true);
    Vehicle& vehicleRef = car; // No slicing
    std::cout << "Max speed: " << vehicleRef.maxSpeed << std::endl;

    delete vPtr;
}
```

Output:

```
General Horn
Turbo Engaged!
Max speed: 250
```

Question 4: Incorrect this Pointer Return Type

Bug and root cause

this is a pointer to the current object (WindowBox* in this class); *this is the object itself as an lvalue, so returning *this as WindowBox& supports dot-based chaining. setWidth returns WindowBox*, therefore box.setWidth(800).setHeight(600) attempts to call setHeight using a pointer with dot syntax, which is invalid.

Corrected code

```
#include <iostream>

class WindowBox {
private:
```

```
    int width;
    int height;
public:
    WindowBox() : width(0), height(0) {}

    WindowBox& setWidth(int width) {
        this->width = width;
        return *this;
    }

    WindowBox& setHeight(int height) {
        this->height = height;
        return *this;
    }

    void display() const {
        std::cout << "Window: " << width
                  << " x " << height << std::endl;
    }
};

int main() {
    WindowBox box;
    box.setWidth(800).setHeight(600);
    box.display();
}
```

Output:

```
Window: 800 x 600
```

Question 5: Private Inheritance and Inaccessible Base

Bug and root cause

With private inheritance, public RawSocket members become private members of SecureChannel. They can be called inside SecureChannel, but client code cannot call channel.closePort(). The conversion SecureChannel* to RawSocket* is also inaccessible outside the derived class. Upcasting is generally accessible with public inheritance; with private inheritance it is restricted to suitable member/friend contexts.

Approach A: expose the inherited method using a public using-declaration. This makes closePort() publicly accessible but does not make the base conversion public.

Corrected code using Approach A

```
#include <iostream>

class RawSocket {
public:
    void openPort(int port) {
        std::cout << "Port " << port << " opened." << std::endl;
    }
    void closePort() {
        std::cout << "Port closed." << std::endl;
    }
};

class SecureChannel : private RawSocket {
public:
```

```
    using RawSocket::closePort;

    void establishEncryptedTunnel(int port) {
        openPort(port);
        std::cout << "Tunnel encrypted with TLS 1.3" << std::endl;
    }
};

int main() {
    SecureChannel channel;
    channel.establishEncryptedTunnel(443);
    channel.closePort();
}
```

Output:

```
Port 443 opened.
Tunnel encrypted with TLS 1.3
Port closed.
```

Note: The using declaration exposes closePort only. It does not make RawSocket* socketPtr = &channel; legal under private inheritance.

Quick Revision

```
// Virtual inheritance (diamond)
class Student : virtual public Person { };

// Most-derived class initializes virtual base
TeachingAssistant(...) : Person(...), Student(...), Employee(...) { }

// Derived constructor forwards to base
Sensor(...) : Device(mac), samplingRate(rate) { }

// Fluent chaining
ClassName& setter(...) {
    // update fields
    return *this;
}

// Avoid slicing: use Base& or Base* for derived objects
```

The uploaded Unit IV document contains five debugging questions and five pseudo-code formulation questions. This solution document addresses the five debugging questions, matching the prior unit workflow.